Cryptocurrency Volatility Prediction Using Machine Learning Models (HAR-X, XGBoost, and LightGBM)

Karina Gridneva

Machine Learning for Finance
Research Paper

July 27, 2026

Contents

1 Introduction

Cryptocurrencies, such as Bitcoin, Ethereum or Binance Coin, are a highly popular and specific class of financial assets. They attract investors because of potential high returns, but these high returns are often associated with high volatility. Volatility can be classified into two types: Implied Volatility (IV) and Realized Volatility (RV). Implied Volatility is derived from option prices and reflects market expectations of future uncertainty, while Realized Volatility is computed from historical high-frequency returns. As liquid options markets for cryptocurrencies remain limited, we use Realized Volatility in this research paper.

The high volatility of cryptocurrencies reflects their risky nature, making accurate volatility forecasting essential for risk management. Moreover, predicting the volatility of cryptocurrencies is crucial for building investment strategies based on the obtained predictions.

Volatility series present a number of difficulties for forecasting. First, time series modeling is complicated by autocorrelation and seasonality, which make classical statistical methods less suitable. Second, the relationship between volatility and other variables can be highly nonlinear, so linear models may be insufficient and machine learning approaches become necessary.

The aim of this research paper is to build and compare linear and machine learning models to forecast Bitcoin (BTC) volatility, and perform a post-forecast feature importance analysis. Two research questions are raised in the study:

1. Which type of variables (external or internal) has the greatest impact on the model's predictive ability?
2. Which model type performs best in forecasting BTC realized volatility?

To address these questions, we employ a set of predictors including lagged volatility, on-chain metrics, investor sentiment, and stock market indicators. We find that XGBoost consistently outperforms all linear benchmarks across every sample split, while LightGBM performs similarly well but is occasionally matched or exceeded by Gamma regression. Among linear models, Lasso and Gamma regression are the most competitive, while OLS produces negative predictions in every specification and Ridge does so in the 70/30 split, which is economically meaningless for volatility forecasting.

2 Data

For the analysis, we use one cryptocurrency: Bitcoin (BTC). Table 1 provides a detailed description of all the data used in this paper along with their sources. All data are taken from open sources. Closing prices and trading volumes are presented relative to Tether (USDT), a stablecoin pegged to the US dollar at a one-to-one ratio. The selection of variables and their classification into internal and external determinants follows Wang et al. [2023].

Table 1: Data and sources.

Category	Data	Frequency	Source
Cryptocurrency data	BTC closing price, trading volume, trade count	Hourly + Daily	CryptoDataDownload
Blockchain & Investor sentiment	Average confirmation time, hash rate, difficulty, transactions per block, average block size	Daily	Blockchain.com
	Crypto Fear & Greed Index	Daily	Alternative.me
Stock market	S&P500 closing price, S&P500 trading volume, VIX closing price	Daily	Yahoo Finance
Engineered features	RV lags (1, 2, 3 days)	Daily	Computed by author
	Rolling means of RV (7 and 30 days)	Daily	Computed by author

All variables are daily time series covering the period from July 17, 2019 to December 31, 2024. The total number of observations is 1,994. The start date is determined by the availability of on-chain data from Blockchain.com. Hourly cryptocurrency data were used exclusively to compute the daily realized volatility series.

The target variable in this study is the daily Realized Volatility (RV) of Bitcoin, computed from hourly price data following Corsi [2009]:

$$RV_t = 100 \cdot \sqrt{\sum_{i \in \Omega_t} r_{t,i}^2} \tag{1}$$

where $r_{t,i} = \ln(p_{t,i}) - \ln(p_{t,i-1})$ is the hourly log-return, and Ω_t is the set of hourly observations within day t. Scaling by 100 converts returns to percentage units. The forecast horizon throughout this study is one day ahead.

Stock market variables (S&P500 and VIX) contain missing values on weekends and public holidays, as stock exchanges do not operate on these days, while cryptocurrency markets trade continuously. Missing values are filled using the Last Observation Carried Forward (LOCF) principle, so that each missing value is replaced by the most recent available observation. This approach avoids look-ahead bias, as no future information is used to fill gaps. To ensure that all features are available at the time of forecasting, each predictor is lagged by one day.

In addition to the raw variables described in Table 1, the engineered features are summarised in Table 2.

Lags and rolling means capture the well-documented persistence of volatility. The autocorrelation structure of the RV series suggests that past volatility is informative about future volatility.

Table 2: Engineered features.

Category	Features	Description
Lags	RV_{t-1}, RV_{t-2}, RV_{t-3}	Lagged values of RV capturing short-term persistence
Rolling means	$RV^{(7)}$, $RV^{(30)}$	7- and 30-day rolling means capturing medium- and long-term volatility levels

The sample is divided into training and test subsets following the temporal order of observations. The first 80% of observations (1,596 days) constitute the training set, used for model estimation and hyperparameter tuning. The remaining 20% (398 days) form the test set, used exclusively for out-of-sample evaluation. This strict temporal split ensures that no future information leaks into the training process. To examine the sensitivity of the results to the choice of training period and test set size, two alternative splits are also considered: a post-COVID split where the training set starts from January 2021 to exclude the COVID-19 shock, and a 70/30 split with a larger test set.

Figure 1 plots the RV series alongside BTC closing prices over the sample period. Notable spikes correspond to the COVID-19 market crash in March 2020 and the FTX collapse in November 2022, two of the most significant events in the cryptocurrency market during the sample period.

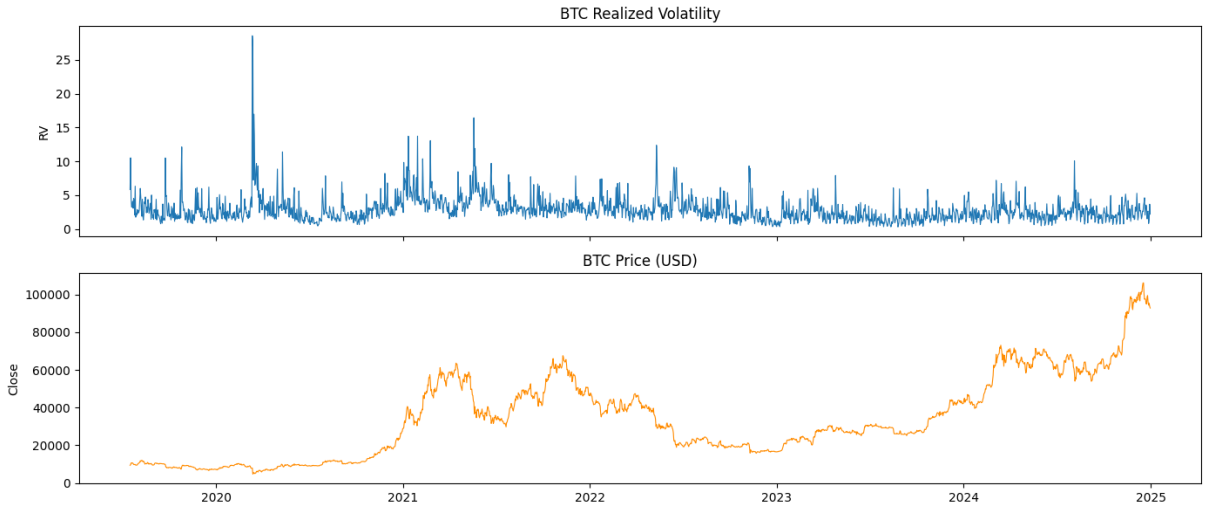

Figure 1: BTC RV and closing price.

Figure 2 shows the distribution of RV and log(RV). The RV series is strongly right-skewed, with the majority of observations concentrated at low values and a long right tail corresponding to extreme volatility episodes. After log-transformation, the distribution becomes approximately symmetric, motivating the use of Gamma regression with a log link function rather than standard OLS.

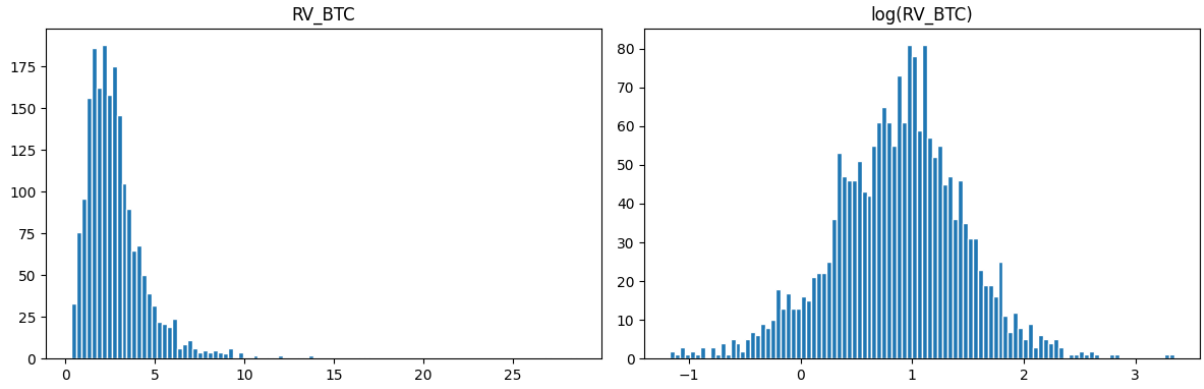

Figure 2: Distribution of RV and log(RV).

3 Methodology

For the analysis, we use six models: HAR-X (OLS, Ridge, Lasso, Gamma), XGBoost, and LightGBM.

As a linear benchmark, we estimate the HAR-X model, an extension of the classical HAR-RV model proposed by Corsi [2009]. While the original HAR-RV regresses realized volatility on its own lags at daily, weekly, and monthly horizons, HAR-X complements this specification with exogenous predictors. The model is estimated using four methods: OLS, Ridge, Lasso, and Gamma regression, allowing us to assess the effect of regularization and distributional assumptions on forecast performance.

OLS minimizes the sum of squared residuals and has an analytical solution requiring no hyper-parameter tuning:

$$RV_{t+1} = c + \sum_{j=1}^{17} \beta_j X_{j,t} + \varepsilon_{t+1} \tag{2}$$

However, OLS places no restrictions on predictions, which can be negative and consequently an economically meaningless outcome for volatility.

Ridge adds an L2 penalty to the OLS objective, shrinking all coefficients toward zero to reduce overfitting while retaining all predictors:

$$\hat{\beta}_{Ridge} = \arg\min_{\beta} \left\{ \|y - X\beta\|^2 + \alpha \|\beta\|_2^2 \right\} \tag{3}$$

Lasso adds an L1 penalty, setting some coefficients exactly to zero and performing automatic feature selection:

$$\hat{\beta}_{Lasso} = \arg\min_{\beta} \left\{ \|y - X\beta\|^2 + \alpha\|\beta\|_1 \right\} \tag{4}$$

Both Ridge and Lasso require tuning of the regularization parameter α, which is selected via cross-validation as described below. Like OLS, neither model guarantees positive predictions.

Gamma regression addresses the negativity problem directly by using a log link function, ensuring predictions are always positive by construction:

$$\log\left(\mathbb{E}[RV_{t+1}]\right) = c + \sum_{j=1}^{17} \beta_j X_{j,t} \tag{5}$$

It also assumes a Gamma distribution for the target variable, which is better suited to the right-skewed nature of RV than the normality assumption underlying OLS.

We include two gradient boosting models, XGBoost [Chen and Guestrin, 2016] and LightGBM [Ke et al., 2017], as the main machine learning alternatives to the linear benchmark. Gradient boosting is an ensemble method that iteratively fits regression trees to the residuals of previous trees:

$$\hat{y}_t = \sum_{k=1}^{K} \eta \cdot f_k(x_t) \tag{6}$$

where η is the learning rate, f_k is the k-th regression tree, and K is the total number of trees. This captures non-linear relationships between predictors and realized volatility that linear models may fail to detect. Both models use a gamma objective function, ensuring positive predictions consistent with the strictly positive nature of RV. XGBoost and LightGBM apply L1 and L2 regularization on tree leaf weights (`reg_alpha` = 0.5, `reg_lambda` = 0.5) and each tree is built on a random subset of 80% of features (`colsample_bytree` = 0.8); these three parameters were fixed at commonly used values and were not included in the hyperparameter search described below. XGBoost grows trees level-wise while LightGBM grows trees leaf-wise. Including both allows us to assess whether the superiority of gradient boosting over linear models is robust to the choice of specific algorithm.

All models with tunable hyperparameters are estimated using `GridSearchCV` with `TimeSeriesSplit` cross-validation with four folds. For XGBoost and LightGBM, the search grid covers the number of estimators, the learning rate, and the maximum tree depth. Standard k-fold cross-validation randomly shuffles observations and is therefore inappropriate for time series, as it allows future observations to appear in the training fold and introduces look-ahead bias. `TimeSeriesSplit` preserves the temporal ordering: each subsequent fold expands the training set while the validation set always follows the training set in time:

Fold 1: $[\text{train}_1][\text{val}_1]$

Fold 2: $[\text{train}_1 + \text{val}_1][\text{val}_2]$

Fold 3: $[\text{train}_1 + \text{val}_1 + \text{val}_2][\text{val}_3]$

Fold 4: $[\text{train}_1 + \text{val}_1 + \text{val}_2 + \text{val}_3][\text{val}_4]$

The optimal hyperparameters are selected by minimizing the mean squared error across the four validation folds; this criterion was applied uniformly across all six models to keep the tuning procedure comparable, and its implications are discussed further in Section 5. Once the best hyperparameters are identified, the model is retrained on the full training set and evaluated on the test set. Feature scaling via `MinMaxScaler` is applied to all linear models prior to training, fitted exclusively on the training set to prevent any information from the test set from influencing the scaling parameters.

4 Results

To evaluate predictions, we use three metrics standard in the volatility forecasting literature [Patton, 2011]: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Quasi-Likelihood loss (QLIKE):

$$RMSE = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(RV_t - \hat{RV}_t)^2} \tag{7}$$

$$MAE = \frac{1}{N}\sum_{t=1}^{N}|RV_t - \hat{RV}_t| \tag{8}$$

$$QLIKE = \frac{1}{N}\sum_{t=1}^{N}\left(\frac{RV_t}{\hat{RV}_t} - \log\frac{RV_t}{\hat{RV}_t} - 1\right) \tag{9}$$

RMSE penalizes large errors more heavily due to the squared term, making it sensitive to extreme volatility episodes. MAE treats all errors equally and is therefore more robust to outliers. QLIKE is specific to volatility forecasting and penalizes underprediction more severely than overprediction, which is consistent with the asymmetric costs of risk underestimation in practice.

To benchmark model performance against a naive forecast, we also report scaled versions of each metric (RMSSE, MASE, QLIKES), computed as the ratio of the model's error to the error

of a random walk forecast $\hat{RV}_{t+1} = RV_t$. A scaled metric below 1 indicates that the model outperforms the naive benchmark.

Table 3 reports forecast performance across all six models and three sample splits. The discussion focuses on RMSSE and QLIKES as they provide the most interpretable comparison against the naive benchmark and capture the asymmetric nature of volatility forecast errors respectively.

Table 3: Forecast performance across model specifications and sample splits.

Model	**RMSE**	**RMSSE**	**MAE**	**MASE**	**QLIKE**	**QLIKES**
Main split (80/20)						
OLS	1.5510	1.1618	1.1343	1.1139	—	—
Ridge	1.4107	1.0567	1.0236	1.0052	0.2538	1.5856
Lasso	1.1631	0.8712	0.8334	0.8184	0.1081	0.6751
Gamma	1.3349	0.9999	0.9464	0.9294	0.1611	1.0065
XGBoost	**1.1403**	**0.8541**	**0.8317**	**0.8168**	**0.1041**	**0.6501**
LightGBM	1.1434	0.8564	0.8493	0.8341	0.1043	0.6518
Post-COVID split (train from 2021)						
OLS	1.5912	1.1918	1.1729	1.1518	—	—
Ridge	1.4548	1.0897	1.0594	1.0404	0.2762	1.7252
Lasso	1.2971	0.9715	0.9193	0.9028	0.1539	0.9615
Gamma	1.4173	1.0616	1.0197	1.0014	0.2017	1.2602
XGBoost	1.2446	0.9322	0.8756	0.8599	0.1348	0.8418
LightGBM	**1.1611**	**0.8697**	**0.8234**	**0.8086**	**0.1082**	**0.6757**
Alternative split (70/30)						
OLS	1.9069	1.4714	1.4475	1.5071	—	—
Ridge	1.7494	1.3498	1.3245	1.3790	—	—
Lasso	1.2113	0.9346	0.9790	1.0193	0.1508	0.7643
Gamma	1.1692	0.9022	**0.8520**	**0.8871**	0.1432	0.7258
XGBoost	**1.1501**	**0.8874**	0.8895	0.9262	**0.1388**	**0.7035**
LightGBM	1.2280	0.9475	0.9935	1.0345	0.1554	0.7877

Note: Bold indicates the best-performing model for each metric within a split.
"—" indicates that QLIKE is not defined due to negative predictions.

One of the two gradient boosting models achieves the lowest RMSSE and QLIKES of all six models in every split: XGBoost in the main and 70/30 splits, and LightGBM in the post-COVID split, confirming that non-linear relationships between predictors and realized volatility exist and can be exploited by tree-based methods. However, in the 70/30 split LightGBM is itself outperformed not only by XGBoost but also by Gamma regression, indicating that the advantage of gradient boosting over linear models is not uniform across specifications.

Among linear models, OLS produces negative predictions in all three splits, leaving QLIKE undefined. Ridge produces negative predictions only in the 70/30 split, and achieves QLIKES above 1.5 in the remaining splits, indicating poor performance. Lasso avoids negative predictions and achieves RMSSE of 0.87 in the main split, but deteriorates to 0.97 in the post-COVID

split and 0.93 in the 70/30 split. Gamma regression guarantees positive predictions through its log link function and achieves RMSSE below 1 in the main split (0.99) and the 70/30 split (0.90), but deteriorates in the post-COVID split with RMSSE of 1.06 and QLIKES of 1.26, suggesting sensitivity to the size of the training set.

In the main split, XGBoost achieves the best results with RMSSE of 0.85 and QLIKES of 0.65, narrowly ahead of LightGBM (RMSSE 0.86, QLIKES 0.65). In the post-COVID split, LightGBM outperforms XGBoost (RMSSE 0.87 vs 0.93, QLIKES 0.68 vs 0.84). In the 70/30 split, XGBoost remains the best model overall (RMSSE 0.89, QLIKES 0.70), but LightGBM (RMSSE 0.95) falls behind not only XGBoost but also Gamma regression (RMSSE 0.90), making it the weakest of the four positive-prediction models in this split. This indicates that while XGBoost's advantage over linear models is robust across specifications, LightGBM's relative ranking is more sensitive to the choice of sample split.

In the post-COVID split, all models perform slightly worse, as expected given the smaller training set. In the 70/30 split, the divergence between linear and non-linear models becomes less clear-cut than in the other two splits: OLS and Ridge still fail to produce valid QLIKE estimates, but Gamma regression closes much of the gap with the boosting models, outperforming LightGBM on both RMSSE and QLIKES.

Figure 3 reports the top-15 features by importance for XGBoost and LightGBM. Feature importance for XGBoost reflects the average gain in predictive performance per split (the scikit-learn default for this library), whereas for LightGBM it reflects split frequency (the scikit-learn default for this library). The two measures are therefore not on a directly comparable numerical scale across models; within-model rankings remain valid and are the basis of the comparisons below.

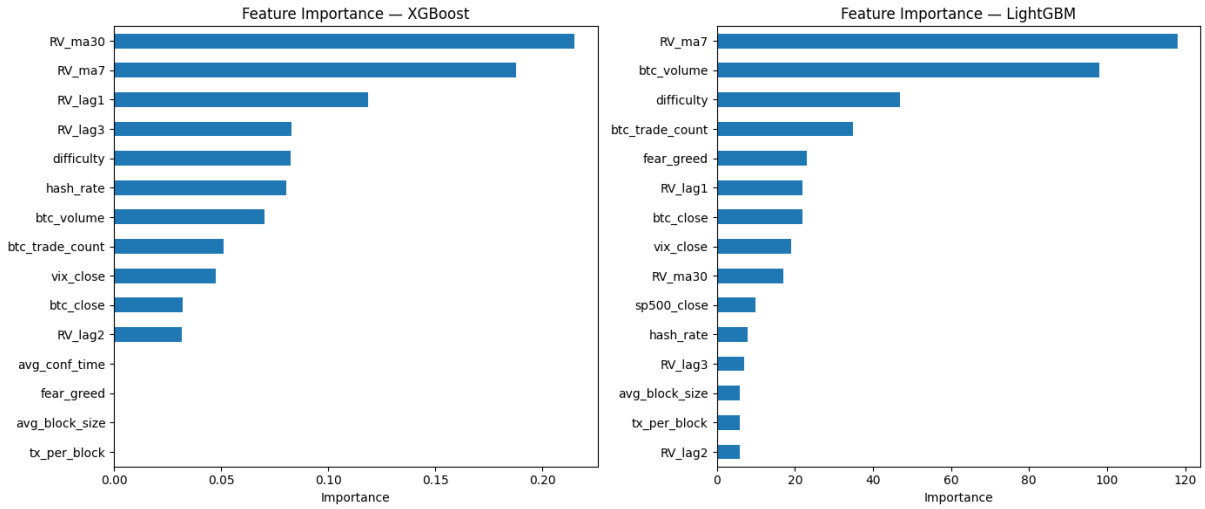

Figure 3: Top-15 features by importance: XGBoost and LightGBM.

Both models assign the highest importance to internal features. In XGBoost, the top four predic-

tors are all internal volatility features: RV_ma30, RV_ma7, RV_lag1, and RV_lag3, confirming the dominance of volatility persistence in driving forecasts. In LightGBM, RV_ma7 ranks first, but btc_volume and btc_trade_count rank second and fourth respectively, suggesting that LightGBM assigns greater weight to trading activity relative to XGBoost.

The two models show notable differences in the lower part of their rankings. In XGBoost, the bottom four features (avg_conf_time, fear_greed, avg_block_size, tx_per_block) have importance values close to zero, meaning the model effectively does not use them despite their formal inclusion. avg_conf_time is similarly negligible in LightGBM, falling outside its top-15 features altogether. However, fear_greed, avg_block_size, and tx_per_block carry non-negligible importance in LightGBM, suggesting that the two algorithms differ in how they exploit sentiment and on-chain network metrics.

Among external features, difficulty and vix_close appear in the top-10 of both models, indicating that blockchain network conditions and broader financial market uncertainty carry incremental predictive value. The Fear and Greed Index ranks fifth in LightGBM but near zero in XGBoost, suggesting its predictive value is model-dependent.

These results provide a partial answer to the first research question: internal determinants, particularly rolling means and lagged volatility, are the primary drivers of forecast performance. External features contribute incrementally but their importance varies across models, with on-chain metrics and stock market variables being the most consistently informative.

From a financial perspective, a model that consistently outperforms the naive random walk benchmark provides actionable value for risk management and portfolio construction. Accurate volatility forecasts allow investors to better size positions and price cryptocurrency derivatives. The superior performance of gradient boosting models, particularly their lower QLIKE, is especially relevant in this context: as QLIKE penalizes underprediction more severely, models with lower QLIKE are less likely to underestimate tail risk, which is critical for managing the extreme volatility episodes that characterize Bitcoin markets.

5 Critical Reflection

The analysis is limited to a single asset, Bitcoin, which restricts the generalization of the findings. Extending the analysis to multiple cryptocurrencies and incorporating their internal features would likely improve forecast performance and allow for more general conclusions.

The set of benchmarks is also limited to linear (HAR-X) and tree-based machine learning models. The volatility forecasting literature uses GARCH-family models, such as GARCH(1,1) or EGARCH, as an additional econometric benchmark. Including such a model would allow for a more complete comparison between classical volatility and the machine learning models.

Data collection itself was difficult. Hourly and daily BTC price data could not be retrieved through a stable API and had to be downloaded manually as CSV files. It means that it is not automatically reproducible and would need to be repeated by hand if the sample period were extended. The on-chain metrics from Blockchain.com's public charts API were easier to automate, but at least one series (average confirmation time) turned out to be considerably noisier than expected, with a small number of days showing extreme values. This was only noticed during a later review of the data, but it did not affect the results, since both gradient boosting models assigned this feature near-zero importance.

The sample period is constrained by the availability of on-chain data from Blockchain.com, which limits the start date to July 2019. A longer sample would increase the number of training observations and potentially improve model stability, particularly for Gamma regression, which showed sensitivity to training set size in the post-COVID split.

The initial modeling approach considered only OLS as the linear benchmark, in line with the standard HAR-RV specification. Early experimentation showed that OLS frequently produced negative volatility forecasts, which motivated adding Ridge and Lasso as regularized alternatives, and ultimately Gamma regression, whose log link function enforces positivity by construction. Ridge, however, did not fully resolve the problem: it still produced negative forecasts in the 70/30 split, which only became apparent once QLIKE was computed. RMSE and MAE alone would not have revealed this failure mode, which suggests that relying on a single evaluation metric can mask economically important problems.

The ranking of models shows some sensitivity to the choice of sample split: XGBoost outperforms LightGBM in the main and 70/30 splits, while LightGBM leads in the post-COVID split; in the 70/30 split, LightGBM is in turn outperformed by Gamma regression. Lasso is competitive in the main split but deteriorates in others. This instability suggests that conclusions about model ranking should be interpreted with caution and may not generalize beyond the specific sample period considered.

The hyperparameter search in this study was conducted over a limited grid. In practice, running `GridSearchCV` with `TimeSeriesSplit` across six models and three sample splits made an exhaustive search impractical within the available time. The grid was narrowed to the ranges reported in Section 3 largely for this practical reason. A more exhaustive search or Bayesian optimization could identify better hyperparameter configurations, particularly for gradient boosting models where the interaction between learning rate, tree depth, and number of estimators is complex.

This study does not include neural network models. Prior literature applying LSTM alongside gradient boosting methods finds that LSTM performs as well as XGBoost and LightGBM [Brauneis and Sahiner, 2024]. It would be interesting to include it as an additional benchmark for a more complete comparison of model classes and to clarify whether its sequential architec-

ture offers any advantage over tree-based methods in this setting.

The comparison of model forecasts relies solely on point metrics without formal statistical testing of whether performance differences are significant. The Diebold-Mariano test [Diebold and Mariano, 2002] is the standard tool for this purpose in the forecasting literature and would strengthen the conclusions.

Finally, this study uses a fixed train-test split rather than a rolling window evaluation scheme. Rolling window validation provides a more realistic simulation and would be particularly informative given structural breaks such as the COVID-19 shock and the FTX collapse, which may have shifted the relationship between predictors and realized volatility.

6 Conclusion

This paper applies machine learning methods to forecast the daily realized volatility of Bitcoin, using a set of internal and external predictors. Six models are estimated and compared across three sample splits: HAR-X with four estimation methods (OLS, Ridge, Lasso, and Gamma regression), XGBoost, and LightGBM.

The results show that XGBoost consistently outperforms all linear benchmarks on RMSE, RMSSE, QLIKE, and QLIKES across every sample split, and that LightGBM does so in the main and post-COVID splits. In the 70/30 split, however, LightGBM is outperformed not only by XGBoost but also by Gamma regression, indicating that the superiority of gradient boosting over linear models is not universal. Among linear models, Gamma regression proves most suitable for volatility forecasting: it guarantees positive predictions through a log link function, outperforms the naive random walk benchmark in most specifications, and in the 70/30 split even achieves lower MAE and MASE than XGBoost.

With respect to the first research question, feature importance analysis reveals that internal determinants, particularly lagged RV values and rolling means, are the primary drivers of forecast performance. External features, including on-chain metrics and stock market variables, contribute incrementally but consistently across both gradient boosting models.

With respect to the second research question, XGBoost delivers the most consistently strong forecasts across all three splits, while LightGBM is competitive in two of the three. This suggests that the advantage of gradient boosting over linear benchmarks depends to some extent on the specific algorithm and the sample period.

From a practical point, accurate volatility forecasts have direct applications in risk management, position sizing, and the pricing of cryptocurrency derivatives. The lower QLIKE of gradient boosting models is particularly relevant in this context, as it implies a lower tendency to underestimate tail risks, which is critical given the extreme volatility episodes of Bitcoin markets.

References

Alexander Brauneis and Mehmet Sahiner. Crypto Volatility Forecasting: Mounting a HAR, Sentiment, and Machine Learning Horserace. *Asia-Pacific Financial Markets*, pages 1–33, 2024.

Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

Fulvio Corsi. A Simple Approximate Long-Memory Model of Realized Volatility. *Journal of Financial Econometrics*, 7(2):174–196, 2009.

Francis X. Diebold and Roberto S. Mariano. Comparing Predictive Accuracy. *Journal of Business & Economic Statistics*, 20(1):134–144, 2002.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Andrew J. Patton. Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1):246–256, 2011.

Yijun Wang, Galina Andreeva, and Belen Martin-Barragan. Machine learning approaches to forecasting cryptocurrency volatility: Considering internal and external determinants. *International Review of Financial Analysis*, 90:102914, 2023.